

challenge-templates

Challenge Templates

The pkg/userflow package provides 13 challenge template types. Each embeds challenge.BaseChallenge and implements the challenge.Challenge interface, with a platform-specific Execute() method.

All templates share these patterns:

- Constructor accepts an ID string, name, description, dependency slice, an adapter, and platform-specific configuration.
- Execute() reports progress via BaseChallenge.ReportProgress().
- Execute() returns a *challenge.Result with assertions, metrics, and outputs.
- Metrics are recorded in seconds (durations) or counts.

Environment Challenges

EnvironmentSetupChallenge

Executes a user-provided setup function to prepare the test environment (start containers, seed data, configure services). Typically the first challenge in a pipeline with no dependencies.

```
func NewEnvironmentSetupChallenge(  
    id string,  
    setupFunc func(ctx context.Context) error,  
    timeout time.Duration,  
) *EnvironmentSetupChallenge
```

Category: "environment"

Execution flow:

1. Applies timeout to context if non-zero.
2. Calls setupFunc(ctx).
3. Produces one assertion: environment_setup / setup_succeeds.
4. Records metric: setup_duration.

EnvironmentTeardownChallenge

Executes a user-provided teardown function to clean up after testing. Typically the last challenge in a pipeline.

```
func NewEnvironmentTeardownChallenge(  
    id string,  
    teardownFunc func(ctx context.Context) error,  
) *EnvironmentTeardownChallenge
```

Category: "environment"

Execution flow:

1. Calls `teardownFunc(ctx)`.
2. Produces one assertion: `environment_teardown / teardown_succeeds`.
3. Records metric: `teardown_duration`.

API Challenges

APIHealthChallenge

Performs a simple health endpoint check by GETting a path and asserting the HTTP status code.

```
func NewAPIHealthChallenge(  
    id string,  
    adapter APIAdapter,  
    healthPath string,  
    expectedCode int,  
    deps []challenge.ID,  
) *APIHealthChallenge
```

Category: "api"

Execution flow:

1. Calls `adapter.GetRaw(ctx, healthPath)`.
2. Produces one assertion: `status_code` comparing actual vs expected.
3. Records metric: `response_time`.

APIFlowChallenge

Executes a multi-step API flow with optional login, variable extraction, and per-step assertions.

```
func NewAPIFlowChallenge(  
    id, name, description string,
```

```

    deps []challenge.ID,
    adapter APIAdapter,
    flow APIFlow,
) *APIFlowChallenge

```

Category: "api"

Execution flow:

1. If credentials are set, calls adapter.Login() and stores the token.
2. For each step: a. Substitutes {{variable}} placeholders in path and body. b. Dispatches the HTTP method (GET, POST, PUT, DELETE). c. Checks expected status code if configured. d. Extracts variables from the JSON response (ExtractTo). e. Evaluates per-step assertions (status_code, response_contains, not_empty).
3. Records per-step duration metrics and total duration.
4. Stores response bodies in outputs.

Browser Challenges

BrowserFlowChallenge

Executes a browser flow: initializes the browser, navigates to a start URL, and performs each step in sequence.

```

func NewBrowserFlowChallenge(
    id, name, description string,
    deps []challenge.ID,
    adapter BrowserAdapter,
    flow BrowserFlow,
) *BrowserFlowChallenge

```

Category: "browser"

Execution flow:

1. Calls adapter.Initialize(ctx, flow.Config).
2. Calls adapter.Navigate(ctx, flow.StartURL).
3. For each step, dispatches the action (navigate, click, fill, select, wait, assert_visible, assert_text, assert_url, screenshot, evaluate_js).
4. Takes screenshots after steps if step.Screenshot is true.
5. Ensures adapter.Close() is called on exit.
6. Records per-step duration metrics, total duration, steps executed, and screenshot count.

Build Challenges

BuildChallenge

Builds one or more targets via a BuildAdapter.

```
func NewBuildChallenge(  
    id, name, description string,  
    deps []challenge.ID,  
    adapter BuildAdapter,  
    targets []BuildTarget,  
) *BuildChallenge
```

Category: "build"

Execution flow:

1. For each target, calls adapter.Build(ctx, target).
2. Produces one build_succeeds assertion per target.
3. Records per-target build duration metrics.

UnitTestChallenge

Runs test suites via a BuildAdapter and aggregates results.

```
func NewUnitTestChallenge(  
    id, name, description string,  
    deps []challenge.ID,  
    adapter BuildAdapter,  
    targets []TestTarget,  
) *UnitTestChallenge
```

Category: "test"

Execution flow:

1. For each target, calls adapter.RunTests(ctx, target).
2. Produces one all_tests_pass assertion per suite.
3. Records total_tests and total_failures metrics.

LintChallenge

Runs linters via a BuildAdapter and reports per-tool results.

```
func NewLintChallenge(  
    id, name, description string,  
    deps []challenge.ID,  
    adapter BuildAdapter,  
    targets []LintTarget,  
) *LintChallenge
```

Category: "lint"

Execution flow:

1. For each target, calls `adapter.Lint(ctx, target)`.
2. Produces one `lint_passes` assertion per tool.
3. Records per-tool warning and error count metrics.

Mobile Challenges

MobileLaunchChallenge

Installs, launches, and verifies stability of a mobile application.

```
func NewMobileLaunchChallenge(  
    id, name, description string,  
    deps []challenge.ID,  
    adapter MobileAdapter,  
    appPath string,  
    stabilityWait time.Duration,  
) *MobileLaunchChallenge
```

Category: "mobile"

Execution flow:

1. Calls `adapter.InstallApp(ctx, appPath)` -- assertion: install.
2. Calls `adapter.LaunchApp(ctx)` -- assertion: launch.
3. Waits for `stabilityWait` duration.
4. Calls `adapter.IsAppRunning(ctx)` -- assertion: stability.
5. Takes a screenshot.
6. Calls `adapter.StopApp(ctx)`.
7. Records `launch_duration` metric.

MobileFlowChallenge

Executes a sequence of mobile steps (tap, send keys, press key, etc.).

```
func NewMobileFlowChallenge(  
    id, name, description string,  
    deps []challenge.ID,  
    adapter MobileAdapter,  
    flow MobileFlow,  
) *MobileFlowChallenge
```

Category: "mobile"

Execution flow:

1. For each step, dispatches the action (launch, tap, send_keys, press_key, wait, screenshot, assert_running, stop).
2. Evaluates per-step assertions.
3. Records per-step duration metrics and steps executed count.

InstrumentedTestChallenge

Runs on-device instrumented tests via a MobileAdapter.

```
func NewInstrumentedTestChallenge(  
    id, name, description string,  
    deps []challenge.ID,  
    adapter MobileAdapter,  
    testClasses []string,  
) *InstrumentedTestChallenge
```

Category: "mobile"

Execution flow:

1. For each test class, calls adapter.RunInstrumentedTests(ctx, cls).
2. Produces one instrumented_tests assertion per class.
3. Records total_tests and total_failures metrics.

Desktop Challenges

DesktopLaunchChallenge

Launches a desktop application, waits for its window, verifies stability.

```
func NewDesktopLaunchChallenge(  
    id, name, description string,  
    deps []challenge.ID,  
    adapter DesktopAdapter,  
    appConfig DesktopAppConfig,  
    stabilityWait time.Duration,  
) *DesktopLaunchChallenge
```

Category: "desktop"

Execution flow:

1. Calls adapter.LaunchApp(ctx, config) -- assertion: launch.
2. Calls adapter.WaitForWindow(ctx, 10s) -- assertion: window.
3. Waits for stabilityWait duration.
4. Calls adapter.IsAppRunning(ctx) -- assertion: stability.

5. Takes a screenshot.
6. Ensures `adapter.Close()` is called on exit.
7. Records `launch_duration` metric.

DesktopFlowChallenge

Executes a `BrowserFlow` in the context of a desktop application's `WebView`.

```
func NewDesktopFlowChallenge(  
    id, name, description string,  
    deps []challenge.ID,  
    adapter DesktopAdapter,  
    flow BrowserFlow,  
) *DesktopFlowChallenge
```

Category: "desktop"

Execution flow:

1. Calls `adapter.Navigate(ctx, flow.StartURL)`.
2. For each step, dispatches the action (navigate, click, fill, wait, `assert_visible`, screenshot).
3. Evaluates per-step assertions.
4. Records per-step duration, total duration, steps executed, screenshot count.

DesktopIPCChallenge

Tests IPC commands sent to the desktop application's backend.

```
func NewDesktopIPCChallenge(  
    id, name, description string,  
    deps []challenge.ID,  
    adapter DesktopAdapter,  
    commands []IPCCommand,  
) *DesktopIPCChallenge
```

Category: "desktop"

Execution flow:

1. For each command, calls `adapter.InvokeCommand(ctx, cmd.Command, cmd.Args...)`.
2. If `ExpectedResult` is set, checks if the response contains the expected string.
3. If no expected result, checks for no error.
4. Evaluates per-command assertions (`response_contains`, `not_empty`).
5. Records per-command duration and `commands_executed` metric.
6. Stores command responses in outputs.